

Git-Vista Feature Matrix

Historical document — planning baseline, not current status. Written 2026-07-16, before the V1 scope freeze (ADR 0049, 2026-08-05, docs/adr/0049-v1-scope-freeze.md). Cell values below (Target, Later) predate the freeze; treat this table as a snapshot of intent on that date, not a live status board. ./dev roadmap and docs/adr/ reflect current shipped/open/cut state.

Rows whose "V2 professional" / "Later" targets ADR 0049 cut (closed won't-do, reopenable): - **Forgejo integration** row — "Target" at V2 traces to #90, cut (depended on #88, also cut). - **GitLab integration** row — "Read-only target if capacity" traces to #91, cut. - **Interactive rebase** row — the touch-plan "Target" at V2 traces to the planner/executor pair #82/#83, both cut. The row's "V1 foundation: Operation foundation" and "Later: Advanced automation" cells are unaffected (no dependency on #82/#83). - **Interactive lessons** row — "First lessons" at V2 and "Lesson marketplace" at Later trace to the simulator (#93) and trainers (#94), both cut. - **Classroom/assessment** row — "Separate service mode" at Later traces to #97 (M7, retired) and #93/#94/#95 (M6 teaching-backend line, cut). - **Forge plugin boundary** row — "Out-of-process SDK" at Later traces to #96, cut (its own precondition, a second real adapter, was never met since #90/#91 are also cut). - **Offline simulator/lessons** row — "Target if capacity" at V2 traces to the same cut simulator/trainer line (#93/#94). - **Paired HTTPS LAN mode** row — "Design only" at V1, "Optional" at V2 traces to #98, cut.

Explain mode row is NOT cut — #92 is explicitly kept; its "Event vocabulary" (V1) / "Target" (V2) cells still describe live scope.

Fetch/pull/push/upstream row is SHIPPED, ahead of this table's cell values — #73 (M2.20) landed and closed out (#234). Fetch, pull, push, and upstream management work over the SSH tunnel today: typed operations with progress streaming and cancellation, an explicit pull strategy (no silent merge/rebase default), force-with-lease-only force publishing (no bare --force/-f path), and idempotent reconnect/retry. Credentialed HTTPS remotes are explicitly **not** shipped — auth reuses the host's Git credential helper and SSH agent only; there is no in-app HTTPS credential prompt. The Current cell below is updated to Yes for the SSH path; the V1 foundation/V2 professional cells stay for HTTPS credential support and multi-remote polish, which remain future work.

GitHub pull requests row is NOT cut — #89 is kept (re-scoped to absorb what it needs from #88).

Status: planning baseline

This matrix separates current implementation, the safe-client foundation, the professional V2 target, and later expansion. It is not a release promise. A feature counts as complete only when its touch UX, operation lifecycle, recovery, security, tests, and documentation are complete.

Legend: Yes implemented, Partial incomplete or narrow, Target planned for that horizon, and Later intentionally deferred.

Product Capabilities

Capability	Current	V1 foundation	V2 professional	Later
Vertical commit graph	Yes	Harden	Semantic zoom and paging	Graph extensions
Commit detail and parent navigation	Yes	Harden	Compare and investigation links	Provider context
Large-history virtualization	Partial	Server paging	Bounded graph streaming	Advanced graph queries
Branch create/delete	Yes	Typed operation	Upstream/protection awareness	Policy plugins
Merge	Partial	Plan and recovery	Conflict lifecycle	Strategy extensions
Clone public URL	Partial	Catalog and SSRF controls	Credentialed clone	Templates
Working-tree status	Partial summary	Complete status view	Refine	-
File/hunk diff	Partial	Target	Large/binary/rename polish	Semantic adapters
Stage/unstage/partial stage	Partial (all only)	Target	Refine	-
Discard changes	No	Safe plan only	Refine	-
Commit/amend/sign	Partial	Target	Signing polish	Enterprise policy
Fetch/pull/push/upstream	Yes (SSH)	Credentialed HTTPS	Multi-remote polish	Provider policy
Tags	Read display	Create/delete	Signed/annotated workflows	-
Stash	No	-	Target	-
Worktrees	No	Foundation model	Target	Cross-host views
Compare refs/commits	No	API foundation	Target	Shareable reports
Cherry-pick/revert	No	-	Target	-
Interactive rebase	No	Operation foundation	Target	Advanced automation
Conflict resolution	No	Conflict state API	Target	Semantic merge adapters
Blame/file history	No	-	Target	Code-host context
Bisect	No	-	Target	Automated test adapters

Capability	Current	V1 foundation	V2 professional	Later
Reflog and recovery	No	Recovery refs/ log	Target	Guided forensics
Operation progress/ cancel	No	Target	Refine	Background notifications
Evidence-based undo	No	Journal/ checkpoints	Target	Provider operation recovery
GitHub pull requests	Deep links only	Provider boundary	Target	Rich review workflows
Forgejo integration	No	Provider boundary	Target	Rich review workflows
GitLab integration	No	Provider boundary	Read-only target if capacity	Rich review workflows
Explain mode	No	Event vocabulary	Target	Personalized explanations
Interactive lessons	No	Sandbox design	First lessons	Lesson marketplace
Classroom/ assessment	No	-	-	Separate service mode

Platform Capabilities

Capability	Current	V1 foundation	V2 professional	Later
Loopback personal mode	Enforced bind; session/CSRF/Host protections	Harden	Supported	-
SSH-tunnel remote Linux	Documented launcher/session workflow	Harden	First-class UX	Managed helper
Paired HTTPS LAN mode	No; insecure HTTP LAN mode removed	Design only	Optional	Device management
Multi-user mode	No	Explicitly excluded	Explicitly excluded	Separate architecture
Touch graph navigation	Yes	Harden	Full app touch UX	Pencil refinements
Portrait/split-screen shell	Partial	Target	Refine	-
Keyboard and trackpad	Partial	Accessible parity	Command palette	Power workflows
Installable PWA	No	App shell/ versioning	Target	Web Push
Offline real-repo reads	No	Cache policy	Optional metadata cache	Encrypted snapshots
Offline real-repo writes	No	Prohibited	Prohibited	Prohibited
Offline simulator/ lessons	No	-	Target if capacity	Full curriculum
Versioned API	No	Target	Stable compatibility window	Extension API
Session auth/ CSRF/origin checks	No	Target	Audited	Team identity separately
Allowed repository roots	No	Target	Harden	Team authorization separately
Per-worktree serialization	No	Target	Harden	Multi-user coordination separately
Structured operation log	No	Target	Recovery UI	Export/teaching events
Forge plugin boundary	No	Capability model	Built-in adapters	Out-of-process SDK

iPad-Centered Competitive View

This comparison is directional and based on documented public capabilities, not an assertion that every competitor behaves identically in every edition.

Product	Stronger than Git-Vista today	Structural limitation/opportunity for Git-Vista
GitKraken Desktop	Mature graph, rebase, conflicts, worktrees, integrations, polish	Desktop interaction and installation model; Git-Vista can make remote Linux and touch primary
GitButler	Strong branch/workspace experimentation and stacked-work concepts	Git-Vista can stay closer to standard Git semantics and support a self-hosted browser surface
LazyGit	Broad Git operation coverage, speed, keyboard efficiency, terminal deployment	Not finger/Pencil oriented; Git-Vista can provide visual plans and accessible touch workflows
VS Code Git Graph	Convenient editor integration and graph actions	Extension remains desktop/editor centered; Git-Vista can be independent of IDE and host OS
Sourcetree	Familiar free desktop workflows and staging UI	Desktop-only interaction assumptions create room for adaptive browser and remote-Linux design

What Git-Vista Can Do Better

Git-Vista does not currently beat mature clients on professional feature depth. Its defensible opportunities are architectural and interaction-based:

- Operate repositories where they already live on Linux while the UI lives on an iPad, without remote-desktop streaming or copying a working tree to the tablet.
- Use one adaptive frontend on iPad, desktop browsers, classroom displays, and self-hosted machines.
- Make history-changing operations reviewable visual plans with explicit preconditions, recovery checkpoints, and teaching explanations.
- Treat worktrees as touch-friendly workspaces rather than an advanced submenu.
- Use Apple Pencil for precise graph, range, hunk, and annotation interactions while preserving full finger accessibility.
- Add teaching to a real Git operation model, so lessons transfer directly to professional work.

Where GitKraken Remains Stronger

- Breadth and maturity of day-to-day Git operations.
- Conflict resolution, interactive rebase, worktrees, integrations, and edge-case behavior tested across a large installed base.
- Desktop-native credential, filesystem, windowing, and OS integration.

- Product polish, onboarding, support, and established user trust.

Git-Vista should not claim parity until routine workflows no longer require a terminal and destructive workflows have credible recovery behavior.

Evaluation Rules

- Revisit this matrix at each milestone and link completed cells to tests or docs.
- Mark partial features honestly; happy-path support is not full support.
- Compare task completion on iPad, not screenshots or raw feature counts.
- Record repository size, network latency, input method, and recovery result in professional workflow tests.

Competitor References

- GitKraken interface: <https://help.gitkraken.com/gitkraken-desktop/interface/>
- GitKraken interactive rebase: <https://help.gitkraken.com/gitkraken-desktop/interactive-rebase/>
- GitKraken worktrees: <https://support.gitkraken.com/gitkraken-desktop/worktrees/>
- LazyGit: <https://github.com/jesseduffield/lazygit>
- VS Code Git Graph: <https://github.com/mhutchie/vscode-git-graph>